

Llenguatges de Programació, FIB, 12 de gener de 2024

L'examen dura tres hores. Poseu el vostre nom a cada full. Contesteu cada problema en fulls diferents. Es valorarà la concisió, claredat i brevetat a més de la completesa i l'exactitud de les respostes. Feu bona lletra.

1 λ -càlcul

(1'5 punts)

1. Doneu una definició de la funció **flip** de Haskell en λ -càlcul.
2. Recordeu que en λ -càlcul:

$$\begin{aligned}\mathbf{true} &\equiv \lambda xy.x \\ \mathbf{false} &\equiv \lambda xy.y \\ \mathbf{two} &\equiv \lambda sz.s(sz)\end{aligned}$$

Donada la definició

$$\mathbf{even} \equiv \lambda n.n \mathbf{flip} \mathbf{true}$$

comproveu que el resultat d'avaluar **even two** en ordre normal és **true**.

2 Raonament equacional en Haskell

(2 punts)

Recordeu que la funció **foldl** ve donada per:

$$\begin{aligned}\mathbf{foldl} \ f \ e \ [] &= e \\ \mathbf{foldl} \ f \ e \ (x:xs) &= \mathbf{foldl} \ f \ (f \ e \ x) \ xs\end{aligned}$$

1. Doneu una funció *fact1* que calculi el factorial d'un natural n de forma recursiva.
2. Doneu una funció *fact2* que calculi el factorial d'un natural n amb el **foldl**.
3. Doneu (sense necessitat de demostrar) una equació que relacioni les expressions següents:
 - $\mathbf{foldl} \ f \ e \ [1..n]$
 - $\mathbf{foldl} \ f \ e \ [1..n-1]$.
4. Demostreu per raonament equacional que *fact1* i *fact2* són equivalents. Podeu utilitzar la relació del punt anterior com a lema.

3 Inferència de tipus

(2 punts)

Recordeu els tipus d'aquestes funcions habituals:

```
id  :: a → a
const :: a → b → a
(.) :: (b → c) → (a → b) → a → c
```

A més, recordeu que les classes *Functor* i *Applicative* venen definides per

```
class Functor f where
  fmap :: (a → b) → (f a → f b)

class Functor f => Applicative f where
  pure  :: a → f a
  <*> :: f (a → b) → f a → f b
```

També, recordeu que *Maybe* instancia aquestes classes així:

```
instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just x) = Just (f x)

instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  Just f <*> x = fmap f x
```

La llibreria estàndard de Haskell defineix l'operador *<\$* i l'operador **>* d'aquesta forma:

```
(<$)    = fmap . const
x *> y  = (id <$ x) <*> y
```

1. Calculeu (pas a pas) quan valen aquestes expressions:

- 'a' <\$ (Just 2)
- 'a' <\$ Nothing
- (Just 'a') *> (Just 2)

2. Inferiu el tipus complet de l'operador *<\$*. Per a fer-ho, dibuixeu l'arbre de sintaxi abstracta corresponent, anoteu l'arbre amb els tipus de cada node, genereu el conjunt de restriccions d'igualtat de tipus i resoleu les restriccions. No us deixeu els requeriments sobre les classes.

3. Igualment, inferiu el tipus complet de l'operador **>*.

4 Python amb tipus

(2 punts)

Malgrat que Python és un llenguatge de tipus dinàmic, des de fa un temps ofereix la possibilitat d'anotar els tipus de les variables, paràmetres i funcions. En temps d'execució, aquestes anotacions no provoquen cap impacte en la manera que s'executen els programes ni sobre el seu temps d'execució: simplement es consideren com a comentaris. L'objectiu d'aquestes anotacions de tipus és permetre que altres eines, com ara `mypy`, puguin verificar estàticament l'absència d'errors de tipus.

Per exemple, en el següent programa en Python amb tipus, `mypy` diagnostica que la crida a `factorial` és incorrecta abans d'executar el programa (estalviant així un error de tipus en temps d'execució):

```
def factorial (n: int) → int: ...  
print( factorial ([1,2,3]))
```

El tipus de les funcions amb paràmetres de tipus $T1, T2, \dots$ i resultat de tipus R s'escriu `Callable[[T1, T2, ...], R]`. Per exemple, aquesta seria la capçalera d'una funció `foldl`:

```
def foldl (Callable[[B, A], B], B, list [A]) → B: ...
```

on A i B són variables de tipus (iguals que en Haskell).

Considereu aquest fragment de programa:

```
class Persona: ...  
class Empleat(Persona): ...  
class Mecanic(Empleat): ...  
  
def f_persona(p: Persona) → int: ...  
def f_empleat(e: Empleat) → int: ...  
def f_mecanic(m: Mecanic) → int: ...  
  
def g_persona(i: int) → Persona: ...  
def g_empleat(i: int) → Empleat: ...  
def g_mecanic(i: int) → Mecanic: ...  
  
def h1(f: Callable[[Empleat], int]) → None: ...  
def h2(g: Callable[[int], Empleat]) → None: ...
```

Llavors, si s'escriu

```
h1(f_persona)      # 1  
h1(f_empleat)      # 2  
h1(f_mecanic)      # 3 !  
  
h2(g_persona)      # 4 !  
h2(g_empleat)      # 5  
h2(g_mecanic)      # 6
```

`mypy` dona un error de tipus a la línia 3 i un error de tipus a la línia 4.

Utilitzant la informació anterior, contesteu les preguntes següents:

1. Doneu les definicions de constructor de tipus covariant, contravariant i invariant.
2. Digueu si, per a mypy, el constructor de tipus **Callable** és covariant, contravariant i/o invariant *en els paràmetres* i expliqueu perquè.
3. Digueu si, per a mypy, el constructor de tipus **Callable** és covariant, contravariant i/o invariant *en el resultat* i expliqueu perquè.
4. Expliqueu per quina raó té sentit l'error de la línia 3.
5. Expliqueu per quina raó té sentit l'error de la línia 4.

5 Aprenent Scheme amb una IA

(2'5 punts)

Considereu aquesta conversa real (minimament editada) entre una usuària i ChatGPT:



Hola màquina!



Bon dia! Com puc ajudar-te avui?



Voldria aprendre a programar en Scheme.



Genial! Scheme és un llenguatge de programació funcional fascinant. Scheme és un llenguatge de programació funcional i minimalista, derivat de Lisp. Es destaca per la seva elegància i simplicitat, amb una sintaxi simple basada en parèntesis que facilita la manipulació d'expressions i el tractament de funcions com a valors.



Està bé. Ensenya'm una funció en Scheme que calculi el màxim de dos nombres.



Clar, aquí tens un exemple de funció que calcula el màxim de dos números en Scheme:

```
(define (maxim a b)      ; funció que retorna el més gran de dos valors
  (if (> a b)
      a
      b))
```



I com escriuries una funció en Scheme per saber si un nombre és primer o no?



Per determinar si un nombre és primer o no en Scheme, pots escriure una funció que verifiqui si té més d'un divisor a part d'1 i d'ell mateix. Aquí tens una manera de fer-ho:

```
(define (es-primer? n)      ; n és un natural
  (define (divisible? divisor)
    (= (remainder n divisor) 0))

  (define (troba-divisor divisor)
    (if (> divisor (/ n 2))      ; si el divisor arriba a n/2,
        #t                      ; n és primer.
        (if (divisible? divisor) ; sinó, si té un divisor,
            #f                   ; n no és primer.
            (troba-divisor (+ divisor 1)))) ; sinó, continua buscant

  (troba-divisor 2))
```



Explica'm com es crea una llista i com s'afageix un element al principi d'una llista.



En Scheme, la llista buida s'escriu '(). La funció *cons* s'utilitza per construir llistes afegint un element al davant d'una altra llista. Per exemple, per crear una llista amb els elements 1, 2 i 3, pots fer-ho així:

```
(define una-llista (cons 1 (cons 2 (cons 3 '()))))
```



Existeixen operacions com **head** i **tail** de Haskell?



Sí: **head** es diu *car* i **tail** es diu *cdr*.



Moltes gràcies.



De res, estic aquí per ajudar! Espero que l'examen d'LP et vagi molt bé.

Contesteu les preguntes següents:

1. La sintaxi de Scheme es basa, essencialment, en expressions parentitzades. Des del punt de vista de la pragmàtica dels llenguatges de programació, doneu un argument a favor i un argument en contra d'aquesta decisió de disseny.
2. Escriviu, en format ANTLR, una gramàtica per reconèixer programes en Scheme amb els elements donats als exemples.
3. La funció *divisible?* dins de la funció *es-primer?* utilitza la variable *n* de *es-primer*. Com s'anomena aquest concepte als llenguatges de programació?
4. Quin és l'espai (asimptòtic) utilitzat per la funció *es-primer*? Expliqueu perquè.
5. És correcta la implementació de *es-primer?* donada per ChatGPT?
6. Escriviu en Scheme una funció *range* que, donats dos enters *i* i *j*, retorni la llista dels enters de *i* a *j*.
7. Escriviu en Scheme una funció *filter* que, donat un predicat i una llista, retorni una llista amb els elements de la llista donada que compleixen el predicat.
8. Escriviu en Scheme una funció *primers-primers* que, donat un natural *n*, retorni la llista de tots els nombres primers fins a *n* (inclòs). No cal que sigui eficient però sí elegant.